

Bài giảng: Ràng buộc trong CREATE TABLE với MySQL

PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT & Toàn vẹn dữ liệu

Vũ Đức Minh

Trường Đại học Kinh tế Quốc dân (NEU), Hà Nội, Việt Nam

September 23, 2026

Mục tiêu bài giảng

Sau khi hoàn thành bài học này, người học có thể:

- Giải thích được bản chất và vai trò của **Constraint** trong bảo đảm toàn vẹn dữ liệu.
- Khai báo thành thạo khóa chính đơn và khóa chính ghép bằng **PRIMARY KEY**.
- Thiết lập mối quan hệ tham chiếu bảng bằng **FOREIGN KEY ... REFERENCES ...**
- Cấu hình chuẩn xác hành vi tham chiếu: **CASCADE, RESTRICT, SET NULL, NO ACTION**.
- Hiểu rõ cơ chế vận hành và rủi ro khi tạm tắt **FOREIGN_KEY_CHECKS**.
- Áp dụng các ràng buộc nghiệp vụ: **NOT NULL, UNIQUE, CHECK** và **DEFAULT**.
- Đặt tên ràng buộc theo **quy ước chuẩn chuyên nghiệp** để dễ bảo trì schema.

Nội dung bài học

Phần 1: Toàn vẹn & Khóa chính/ngoại

- 1. Toàn vẹn dữ liệu & Khái niệm Constraint
- 2. Quy ước đặt tên ràng buộc chuẩn
- 3. PRIMARY KEY: Khóa đơn & Khóa ghép
- 4. FOREIGN KEY: Quan hệ cha - con
- 5. Hành vi tham chiếu (ON DELETE/UPDATE)
- 6. Kỹ thuật tắt mở FOREIGN KEY CHECKS

Phần 2: Ràng buộc nghiệp vụ & Vận dụng

- 7. NOT NULL: Bắt buộc nhập liệu
- 8. UNIQUE: Chống trùng lặp dữ liệu
- 9. CHECK: Ràng buộc miền giá trị logic
- 10. DEFAULT: Giá trị mặc định tự sinh
- 11. Schema mẫu tổng hợp đa ràng buộc
- 12. Bẫy lỗi thường gặp, Quiz & Bài tập

1. Constraint là gì? Ba mức độ toàn vẹn dữ liệu

Định nghĩa

Constraint (Ràng buộc) là các quy tắc được định nghĩa trên bảng hoặc cột mà Hệ quản trị CSDL (*DBMS*) tự động kiểm tra nhằm bảo đảm dữ liệu luôn **chính xác, hợp lệ và nhất quán**.

Toàn vẹn thực thể

- Mỗi dòng có định danh duy nhất.
- Do **PRIMARY KEY** đảm bảo.
- Cấm trùng lặp và cấm NULL.

Toàn vẹn tham chiếu

- Liên kết các bảng luôn hợp lệ.
- Do **FOREIGN KEY** đảm bảo.
- Chặn con trỏ tới cha không tồn tại.

Toàn vẹn miền giá trị

- Dữ liệu đúng quy tắc nghiệp vụ.
- Do **NOT NULL, CHECK, UNIQUE, DEFAULT** đảm bảo.

2. Quy ước đặt tên Constraint chuẩn chuyên nghiệp

Tại sao nên đặt tên tường minh?

Nếu không chỉ định tên, MySQL tự sinh tên ngẫu nhiên (ví dụ `students_chk_1`). Khi chạy lệnh `ALTER TABLE DROP CONSTRAINT`, việc truy vết và quản lý trong script migration sẽ rất phức tạp!

Loại Constraint	Tiền tố	Cấu trúc mẫu	Ví dụ minh họa
Primary Key	pk_	pk_<ten_bang>	pk_students, pk_order_lines
Foreign Key	fk_	fk_<bang_con>_<bang_cha>	fk_employees_departments
Unique Key	uq_	uq_<bang>_<cot>	uq_users_email, uq_students_code
Check Rule	chk_	chk_<bang>_<nghep_vu>	chk_products_price, chk_users_age

Cú pháp khai báo tường minh

```
CONSTRAINT constraint_name <loai_rang_buoc> (<danh_sach_cot>)
```

3. PRIMARY KEY đơn cột (Single-column Primary Key)

Đặc điểm của PRIMARY KEY

- Giá trị phải **duy nhất tuyệt đối** trong bảng.
- **Không được chứa giá trị NULL.**
- Mỗi bảng **chỉ có duy nhất một** Primary Key.
- Tự động tạo **Clustered Index** (trong InnoDB), giúp tối ưu tốc độ tìm kiếm theo khóa.

```
-- Khai báo inline trên cột
CREATE TABLE departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(100) NOT NULL
);

-- Khai báo table constraint (Khuyến nghị)
CREATE TABLE departments (
    dept_id    INT,
    dept_name  VARCHAR(100) NOT NULL,
    phone      VARCHAR(20),
    CONSTRAINT pk_departments
        PRIMARY KEY (dept_id)
) ENGINE = InnoDB;
```

4. Khóa chính ghép (Composite Primary Key)

Bản chất của Composite Key

- Gồm từ **hai cột trở lên** kết hợp thành.
- Từng cột riêng lẻ có thể lặp lại giá trị.
- Tổ hợp tất cả các cột** trong khóa phải duy nhất.
- Ứng dụng:** Bảng nối quan hệ nhiều - nhiều ($N - N$), ví dụ sinh viên đăng ký môn học theo kỳ.

```
CREATE TABLE enrollments (
  student_id INT,
  course_id INT,
  semester VARCHAR(20),
  enrolled_at DATE,
  CONSTRAINT pk_enrollments
    PRIMARY KEY (student_id, course_id,
  semester)
) ENGINE = InnoDB;

-- Các dòng hợp lệ:
-- (1, 101, '2026A')
-- (1, 102, '2026A') -> student_id lặp lại
-- (1, 101, '2026B') -> semester khác nhau

-- Lỗi Duplicate entry vì trùng 3 cột:
-- (1, 101, '2026A')
```

5. FOREIGN KEY: Quan hệ tham chiếu Cha - Con

Nguyên lý Khóa ngoại

- Cột ở **Bảng con** tham chiếu đến **PRIMARY KEY** hoặc **UNIQUE KEY** ở **Bảng cha**.
- Chặn đứng việc tạo dữ liệu con “mồ côi” (trở đến bản ghi cha không tồn tại).
- Kiểu dữ liệu giữa cột con và cột cha phải tương thích hoàn toàn.

```
-- Bảng cha: departments
CREATE TABLE departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(100) NOT NULL
) ENGINE = InnoDB;

-- Bảng con: employees
CREATE TABLE employees (
    emp_id     INT PRIMARY KEY,
    full_name  VARCHAR(100) NOT NULL,
    dept_id    INT,
    CONSTRAINT fk_employees_departments
        FOREIGN KEY (dept_id)
        REFERENCES departments(dept_id)
) ENGINE = InnoDB;
```

6. Hành vi tham chiếu: ON DELETE & ON UPDATE

Tùy chọn	Khi dòng ở bảng cha bị XÓA (ON DELETE)	Khi khóa ở bảng cha bị SỬA (ON UPDATE)
RESTRICT	Chặn lệnh xóa nếu có ít nhất 1 dòng con đang tham chiếu (mặc định an toàn)	Chặn lệnh sửa khóa cha nếu có dòng con đang liên kết
CASCADE	Tự động xóa luôn tất cả các dòng con liên quan	Tự động cập nhật khóa mới xuống tất cả các dòng con
SET NULL	Đặt cột FK ở con thành NULL (yêu cầu cột con cho phép nhận NULL)	Đặt cột FK ở con thành NULL

Cảnh báo an toàn dữ liệu

Không lạm dụng CASCADE tùy tiện! Chỉ dùng ON DELETE CASCADE khi quan hệ là phụ thuộc sống còn (như xóa Orders thì xóa các dòng trong OrderLines).

7. Quản lý Khóa ngoại bằng ALTER TABLE

Thêm và xóa khóa ngoại trên bảng đang chạy

Trong quy trình phát triển thực tế, bảng thường được tạo trước, sau đó khóa ngoại mới được thêm vào thông qua các script migration.

```
-- 1. Thêm Foreign Key mới vào bảng đã tồn tại
ALTER TABLE employees
ADD CONSTRAINT fk_employees_departments
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
ON UPDATE CASCADE ON DELETE RESTRICT;

-- 2. Xem định nghĩa và tên constraint thực tế
SHOW CREATE TABLE employees;

-- 3. Xóa Foreign Key khi cần thay đổi logic nghiệp vụ
ALTER TABLE employees
DROP FOREIGN KEY fk_employees_departments;
```

Lưu ý

8. Kỹ thuật tắt kiểm tra khóa ngoại: FOREIGN_KEY_CHECKS

Mục đích sử dụng

- Nạp khối lượng lớn dữ liệu (*bulk import*) khi bảng cha và con phụ thuộc vòng tròn.
- Tái tạo nhanh cấu trúc schema trong môi trường lab hoặc staging.
- Truncate bảng cha mà không cần gỡ bỏ quan hệ từng bảng con.

```
-- Kiểm tra trạng thái
SELECT @@FOREIGN_KEY_CHECKS;

-- Tắt kiểm tra khóa ngoại (Session)
SET FOREIGN_KEY_CHECKS = 0;

-- Thao tác dữ liệu đặc biệt
TRUNCATE TABLE departments;

-- Bắt buộc phải bật lại ngay sau đó!
SET FOREIGN_KEY_CHECKS = 1;
```

Rủi ro nghiêm trọng

Khi `FOREIGN_KEY_CHECKS = 0`, MySQL cho phép lưu dữ liệu vi phạm tham chiếu. Khi bật lại (`= 1`), MySQL **không tự rà soát sửa dữ liệu cũ**! Tuyệt đối

9. Ràng buộc NOT NULL: Bắt buộc nhập liệu

Nguyên lý NOT NULL

- Buộc cột phải luôn có giá trị hợp lệ khi INSERT hoặc UPDATE.
- Tránh phát sinh giá trị UNKNOWN trong logic ba ngôi (*three-valued logic*).
- Hỗ trợ tối ưu hóa lưu trữ và đánh chỉ mục.

```
CREATE TABLE users_not_null (
  user_id      INT PRIMARY KEY,
  full_name    VARCHAR(100) NOT NULL,
  email        VARCHAR(150) NOT NULL,
  created_at   DATETIME NOT NULL
) ENGINE = InnoDB;

-- Chen hop le:
INSERT INTO users_not_null VALUES
(1, 'Nguyen An', 'an@neu.vn', NOW());

-- Bao loi 1048 (Column cannot be null):
INSERT INTO users_not_null VALUES
(2, NULL, 'binh@neu.vn', NOW());
```

Phân biệt NULL và Chuỗi rỗng ''

NULL là **chưa có giá trị**. Chuỗi rỗng '' là **chuỗi có chiều dài 0**. Cột NOT NULL vẫn nhận chuỗi rỗng. Muốn cấm chuỗi rỗng, cần kết hợp thêm CHECK

10. Ràng buộc UNIQUE: Đảm bảo tính duy nhất

So sánh UNIQUE vs PRIMARY KEY

- **Số lượng:** Mỗi bảng chỉ có 1 PK, nhưng có thể có **nhiều ràng buộc UNIQUE**.
- **Giá trị NULL:** Trong MySQL, cột UNIQUE cho phép chứa **nhiều giá trị NULL** (trừ khi có thêm NOT NULL).
- Để đảm bảo vừa bắt buộc có giá trị, vừa duy nhất: dùng **NOT NULL UNIQUE**.

```
CREATE TABLE room_bookings (
  booking_id   INT PRIMARY KEY,
  room_id      INT NOT NULL,
  booking_date DATE NOT NULL,
  time_slot    VARCHAR(20) NOT NULL,

  -- UNIQUE to hop 3 cot
  CONSTRAINT uq_room_bookings_slot
    UNIQUE (room_id, booking_date,
            time_slot)
) ENGINE = InnoDB;

-- Cung 1 phong trong 1 ngay tai 1 ca chi
  duoc dat duy nhat 1 lan!
```

11. Ràng buộc CHECK: Miền giá trị logic nghiệp vụ

Đặc điểm của CHECK

- Buộc dữ liệu phải thỏa mãn một biểu thức Boolean (trả về TRUE hoặc UNKNOWN).
- Hỗ trợ thực thi từ **MySQL 8.0.16** trở lên.
- Có thể kiểm tra logic trên 1 cột hoặc **so sánh tương quan giữa nhiều cột** trong dòng.

```
CREATE TABLE promotions (  
    promo_id      INT PRIMARY KEY,  
    promo_name    VARCHAR(100) NOT NULL,  
    start_date    DATE NOT NULL,  
    end_date      DATE NOT NULL,  
    discount_pct  DECIMAL(5,2) NOT NULL,  
  
    -- So sanh giua 2 cot  
    CONSTRAINT chk_promo_dates  
        CHECK (end_date >= start_date),  
  
    -- Mien gia tri phan tram  
    CONSTRAINT chk_promo_discount  
        CHECK (discount_pct BETWEEN 0 AND  
              100)  
    ) ENGINE = InnoDB;
```

12. Ràng buộc DEFAULT: Cung cấp giá trị tự động

Cơ chế hoạt động của DEFAULT

- Tự động điền giá trị quy định nếu lệnh INSERT không cung cấp cột đó.
- Có thể dùng hằng số ('Pending', 0) hoặc hàm thời gian (CURRENT_TIMESTAMP).
- Nếu người dùng truyền giá trị tường minh, giá trị đó sẽ ghi đè giá trị default.

```
CREATE TABLE tasks (
  task_id      INT PRIMARY KEY,
  task_name    VARCHAR(150) NOT NULL,
  status       VARCHAR(20) NOT NULL DEFAULT 'Pending',
  priority     INT NOT NULL DEFAULT 3,
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB;

-- Chi chen task_id va task_name
INSERT INTO tasks (task_id, task_name)
VALUES (1, 'Soan de thi cuoi ky');

-- status tu dong = 'Pending'
-- priority tu dong = 3
-- created_at tu dong = thoi gian hien tai!
```

13. Thực hành: Bản thiết kế đa ràng buộc hoàn chỉnh

```

CREATE TABLE course_registrations (
  student_id    INT NOT NULL,
  course_id     INT NOT NULL,
  semester      VARCHAR(20) NOT NULL DEFAULT '2026A',
  final_score   DECIMAL(4,2),
  registered_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

  -- 1. Composite Primary Key
  CONSTRAINT pk_course_registrations
    PRIMARY KEY (student_id, course_id, semester),

  -- 2. Foreign Key tham chiếu đến bảng sinh viên và môn học
  CONSTRAINT fk_cr_student FOREIGN KEY (student_id) REFERENCES students(student_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_cr_course FOREIGN KEY (course_id) REFERENCES courses(course_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,

  -- 3. Check Constraint kiểm soát điểm số hợp lệ
  CONSTRAINT chk_cr_score CHECK (final_score BETWEEN 0.00 AND 10.00)
) ENGINE = InnoDB;

```

14. Các bẫy lỗi thường gặp với Constraints

Lỗi quan hệ & Thứ tự thực thi

- **Tạo bảng con trước bảng cha:**
Lỗi không tìm thấy cha khi tạo FK. Phải tạo cha trước, con sau.
- **Dùng SET NULL trên cột NOT NULL:** Xung đột logic nghiêm trọng, MySQL sẽ chặn ngay khi định nghĩa.
- **Lạm dụng ON DELETE CASCADE:** Vô tình xóa toàn bộ dữ liệu lịch sử hóa đơn khi người dùng bị xóa.

Lỗi kiểm tra & Cấu hình

- **Quên bật lại FOREIGN_KEY_CHECKS:** Để hệ thống chạy ở trạng thái tắt kiểm tra làm dữ liệu rác tràn vào CSDL.
- **Nghĩ UNIQUE cấm NULL:** Cột UNIQUE không có NOT NULL sẽ cho phép vô số dòng mang giá trị NULL.
- **Chạy CHECK trên MySQL cũ:** Trước bản 8.0.16, MySQL chỉ nhận diện cú pháp nhưng không kiểm tra thật.

15. Câu hỏi ôn tập & củng cố

Câu 1: Số lượng khóa chính

Một bảng trong CSDL quan hệ có thể có tối đa bao nhiêu khóa chính (Primary Key)?

- A. Tùy ý bao nhiêu cũng được
- B. **Duy nhất 1 Primary Key** (Chính xác!)
- C. Tối đa 2 (1 đơn, 1 ghép)
- D. Không bắt buộc giới hạn

Câu 2: Hành vi CASCADE

Khi bảng cha có dòng bị xóa, tùy chọn nào tự động xóa các dòng con tương ứng?

- A. RESTRICT
- B. SET NULL
- C. **CASCADE** (Chính xác!)
- D. NO ACTION

Câu 3: Đặt tên Constraint

Hỏi: Vì sao nên tuân thủ quy ước đặt tên như pk_orders.

Tổng kết bài học

3 Nguyên tắc cốt lõi

- **Chặn lỗi tại tầng Database:**
Không bao giờ phụ thuộc 100% vào validation ở tầng ứng dụng.
- **Minh bạch quan hệ:** Khai báo đầy đủ Foreign Key để bảo toàn liên kết thực thể.
- **Đầy đủ ràng buộc:** Luôn kết hợp linh hoạt NOT NULL, UNIQUE, CHECK và DEFAULT.

Bài học tiếp theo

- Bài tiếp theo: **Quản lý vòng đời Table trong MySQL** (ALTER TABLE, RENAME, DROP, TRUNCATE & Generated Columns).
- Thực hành: Hoàn thành kịch bản lab schema constraints_lab.